

Глава 4

Имитационные модели

Имитационное моделирование как инструмент принятия решений

Имитационное моделирование – один из видов компьютерного моделирования, использующий методологию системного анализа, центральной процедурой которого является построение обобщенной модели, отражающей все факторы реальной системы, в качестве методологии исследования выступает вычислительный эксперимент.

Имитационная модель строится целенаправленно, поэтому для нее характерно адекватное отображение исследуемого объекта, логико-математическая модель системы представляет собой программно реализованный алгоритм функционирования системы. При имитационном моделировании структура моделируемой системы адекватно отображается в модели, а процесс ее функционирования имитируется на построенной модели.

Под **имитацией** понимают проведение на компьютерах различных серий экспериментов с моделями, которые представлены в качестве некоторого набора (комплекса) компьютерных программ. Сравнение характеристик (конструкций, управлений) моделируемого объекта осуществляется путем вариантных просчетов.

Особую роль имеет возможность **многократного** воспроизведения моделируемых процессов с последующей их статистической обработкой, позволяющая учитывать случайные внешние воздействия на изучаемый объект. На основе набираемой в ходе компьютерных экспериментов статистики делаются выводы в пользу того или иного варианта функционирования, или конструкции реального объекта, или сущности явления.

В ряде случаев формировать решения с помощью формальных методов не удастся – **эксперт должен быть включен в процесс принятия решения**. Он становится активным компонентом информационной системы, детализирует проблему и модель, осуществляет постановку направленного вычислительного эксперимента на модели, генерацию и ранжирование альтернатив, выбор критериев для принятия решений.

Имитационное моделирование значительно расширяет возможности и эффективность работы лиц, принимающих решения, предоставляя им удобный инструмент и средства для достижения поставленных целей. Имитационное моделирование реализует итеративный характер разработки модели системы, поэтапный характер детализации моделируемых подсистем, что позволяет постепенно увеличивать полноту оценки принимаемых решений по мере выявления новых проблем и получения новой информации.

Имитационная модель не дает оптимального решения подобно классическому решению задач оптимизации, но она является удобным для системного анализа вспомогательным средством для поиска решения определенной проблемы.

Область применения имитационных моделей практически не ограничена, это могут быть задачи: исследования структур сложных систем и их динамики, анализа узких мест, прогнозирования и планирования и т.д. Главным преимуществом имитационного моделирования является то, что эксперт может ответить на вопрос: **«Что будет, если ... »**, т.е. с помощью эксперимента на модели вырабатывать стратегию развития.

В последнее время ведутся работы по разработке систем, способных оказать помощь эксперту при ответе на обратный вопрос **«Что надо, чтобы ...»**. Это можно назвать **«целевым моделированием»**, при котором на вход системы подаются показатели целевого состояния, а также перечень возможных регуляторов с указанием диапазона и шага их изменения. Система в автоматическом или полуавтоматическом режиме находит сочетание значений этих регуляторов для достижения заданного целевого состояния.

После того как модель построена, в ней выделяются управляемые параметры и выбираются такие значения этих параметров, при которых проблема либо снимается, либо перестает быть критически важной.

Многократное использование имитационной модели при различных реализациях входных случайных величин позволит оценить параметры распределений тех случайных величин, которые представляют наибольший интерес и служат **выходными в имитационной модели**.

Библиотека SimPy

SimPy – это Python фреймворк процессо-ориентированной дискретно-событийной системы моделирования. Его диспетчеры событий основаны на функциях-генераторах Python.

Также они могут использоваться для создания асинхронных сетей или для реализации мультиагентных систем (с как моделируемым, так и реальным взаимодействием).

Процессы в SimPy – это просто Python генераторы, которые используются для моделирования активных компонентов, например, таких как покупатели, транспортные средства или агенты. SimPy также обеспечивает различные виды общих ресурсов для моделирования точек с ограниченной пропускной способностью (например, серверов, касс, тоннелей). Начиная с версии 3.1, SimPy также будет обеспечить возможности мониторинга для помощи в сборе статистических данных о ресурсах и процессах.

Моделирование может выполняться в режиме «as fast as possible», в режиме реального времени (wall clock time) или в режиме ручного выполнения событий.

Теоретически, в SimPy можно сделать непрерывную симуляцию, но фактически это не осуществимо. Однако, в случае моделирования с шагом фиксированного размера, где процессы не взаимодействуют друг с другом или с общими ресурсами, может использоваться простой цикл while.

SimPy распространяется с руководством, подробной документацией и большим количеством примеров.

SimPy выпущен как open source проект под лицензией MIT. Первая версия была выпущена в декабре 2002 года.

Примеры имитационных моделей в SimPy

1: Обслуживание клиентов в банке

В рассматриваемой модели есть один менеджер банка, который обслуживает клиентов, которые приходят в отделение банка с интенсивностью `ARRIVAL_INTENSITY`. В среднем клиенты получают консультацию у менеджера в течение времени `TIME_IN_BANK`. Однако, клиенты являются нетерпеливыми, каждый из них заставляя менеджера занятым готов ждать в течение случайного времени, распределенного равномерно с параметрами `MIN_PATIENCE` и `MAX_PATIENCE`. Если за это время менеджер не освободился, то клиент покидает банк. Моделирование выполняется для `NEW_CUSTOMERS` клиентов.

```
import random
import simpy

RANDOM_SEED = 42
NEW_CUSTOMERS = 5 # Число клиентов
ARRIVAL_INTENSITY = 1 / 10 # Интенсивность новых клиентов
MIN_PATIENCE = 1 # Минимальное терпение клиента
MAX_PATIENCE = 3 # Максимальное терпение клиента
TIME_IN_BANK = 12 # Среднее время в банке

def source(env, number, arrival_intensity, time_in_bank, min_patience,
max_patience, manager):
    """генерирует новых клиентов"""
    for i in range(number):
        t = random.expovariate(arrival_intensity)
        yield env.timeout(t)
        c = customer(env, f'Клиент_{i + 1}', manager, time_in_bank, min_patience,
max_patience)
        env.process(c)
```

```

def customer(env, name, manager, time_in_bank, min_patience, max_patience):
    """Клиенты приходят, обслуживаются и уходят"""
    arrive = env.now
    print(f'{round(arrive,3)}: {name} пришел')

    with manager.request() as req:
        patience = random.uniform(min_patience, max_patience)
        # Пытается дождаться, иначе уходит
        results = yield req | env.timeout(patience)

    wait = env.now - arrive

    if req in results:
        # Дождался
        print(f'{round(env.now, 3)}: {name} ждал {round(wait,3)}')

        tib = random.expovariate(1.0 / time_in_bank)
        yield env.timeout(tib)
        print(f'{round(env.now, 3)}: {name} закончил обслуживание')

    else:
        # Отказ
        print(f'{round(env.now, 3)}: отказался после {round(wait, 3)}')

# Настройка и запуск моделирования
random.seed(RANDOM_SEED)
env = simpy.Environment()

manager = simpy.Resource(env, capacity=1)
env.process(source(env, NEW_CUSTOMERS, ARRIVAL_INTENSITY, TIME_IN_BANK,
MIN_PATIENCE, MAX_PATIENCE, manager))
env.run()

```

Вывод:

```

10.201: Клиент_1 пришел
10.201: Клиент_1 ждал 0.0
10.454: Клиент_2 пришел
12.807: отказался после 2.353
13.232: Клиент_1 закончил обслуживание
23.79: Клиент_3 пришел
23.79: Клиент_3 ждал 0.0
30.366: Клиент_3 закончил обслуживание
46.063: Клиент_4 пришел
46.063: Клиент_4 ждал 0.0
46.365: Клиент_5 пришел
47.418: отказался после 1.053
54.51: Клиент_4 закончил обслуживание

```

2: Автомобильная мойка

В рассматриваемой модели есть NUM_MACHINES автомобильных моек, каждая из которых моет автомобиль в течение времени WASHTIME. Автомобили пребывают на автомойку с интенсивностью ARRIVAL_INTENSITY. Моделирование выполняется в течение времени SIM_TIME.

```
import random
import simpy

RANDOM_SEED = 42
NUM_MACHINES = 2 # Количество машин на автомойке
WASHTIME = 5 # Время мойки машин
ARRIVAL_INTENSITY = 1 / 7 # Интенсивность поступления новых машин на мойку
SIM_TIME = 20 # Время моделирования

class Carwash(object):
    def __init__(self, env, num_machines, washtime):
        self.env = env
        self.machine = simpy.Resource(env, num_machines)
        self.washtime = washtime

    def wash(self, car):
        yield self.env.timeout(self.washtime)

def car(env, name, cw):
    print(f'{round(env.now, 3)}: {name} прибывает на автомойку')

    with cw.machine.request() as request:
        yield request

    print(f'{round(env.now, 3)}: {name} занимает автомойку')
    yield env.process(cw.wash(name))

    print(f'{round(env.now, 3)}: {name} освобождает автомойку')

def setup(env, num_machines, washtime, arrival_intensity):
    # Генерация автомоек
    carwash = Carwash(env, num_machines, washtime)

    i = 0
    # Генерация новых автомобилей
    while True:
        yield env.timeout(random.expovariate(arrival_intensity))
        i += 1
        env.process(car(env, f'Автомобиль_{i}', carwash))

# Настройка и запуск моделирования
random.seed(RANDOM_SEED)
```

```

env = simpy.Environment()
env.process(setup(env, NUM_MACHINES, WASHTIME, ARRIVAL_INTENSITY))

env.run(until=SIM_TIME)

```

Вывод:

```

7.14: Автомобиль_1 прибывает на автомойку
7.14: Автомобиль_1 занимает автомойку
7.318: Автомобиль_2 прибывает на автомойку
7.318: Автомобиль_2 занимает автомойку
9.569: Автомобиль_3 прибывает на автомойку
11.337: Автомобиль_4 прибывает на автомойку
12.14: Автомобиль_1 освобождает автомойку
12.14: Автомобиль_3 занимает автомойку
12.318: Автомобиль_2 освобождает автомойку
12.318: Автомобиль_4 занимает автомойку
17.14: Автомобиль_3 освобождает автомойку
17.318: Автомобиль_4 освобождает автомойку

```

3: Завод по производству деталей

В этой модели есть завод, на котором NUM_MACHINES станков, каждый из которых производит детали в течение времени, распределенного по нормальному закону с параметрами PT_MEAN и PT_SIGMA (мин). Однако, станок может выходить из строя через случайное время со средним MTTF (мин). В случае поломки станок требует ремонтных работ, которые осуществляет один ремонтник в течение времени REPAIR_TIME (мин). Моделирование выполняется в течение 4 недель. Требуется посчитать сколько деталей произведет каждый станок за это время.

```

import random
import simpy

RANDOM_SEED = 42
PT_MEAN = 10.0      # Среднее время производства детали (мин)
PT_SIGMA = 2.0      # СКО времени производства детали (мин)
MTTF = 300.0        # Среднее время до поломки станка (мин)
BREAK_MEAN = 1 / MTTF # Параметр экспоненциального распределения
REPAIR_TIME = 30.0   # Время восстановления станка (мин)
NUM_MACHINES = 10    # Число станков на заводе
WEEKS = 4            # Время моделирования (нед)
SIM_TIME = WEEKS * 7 * 24 * 60 # Время моделирования (мин)

def time_per_part():
    """Возвращает время выполнения одной конкретной детали"""
    t = random.normalvariate(PT_MEAN, PT_SIGMA)
    # Нормальное распределение может дать отрицательную величину, нам нужна
    только положительная
    while t <= 0:
        t = random.normalvariate(PT_MEAN, PT_SIGMA)
    return t

```

```

def time_to_failure():
    """Возвращает время до следующей поломки станка"""
    return random.expovariate(BREAK_MEAN)

class Machine(object):
    def __init__(self, env, name, repairman):
        self.env = env
        self.name = name
        self.parts_made = 0
        self.broken = False

        # Начать процессы работы и поломки станка
        self.process = env.process(self.working(repairman))
        env.process(self.break_machine())

    def working(self, repairman):
        while True:
            # Начать делать новые детали
            done_in = time_per_part()
            while done_in:
                start = self.env.now
                try:
                    yield self.env.timeout(done_in)
                    done_in = 0

                except simpy.Interrupt:
                    self.broken = True
                    done_in -= self.env.now - start # Как много времени осталось для
                    ВЫПОЛНЕНИЯ

                    # Попросить ремонтника починить станок
                    with repairman.request() as req:
                        yield req
                        yield self.env.timeout(REPAIR_TIME)

                    self.broken = False

            # Деталь выполнена
            self.parts_made += 1

    def break_machine(self):
        while True:
            yield self.env.timeout(time_to_failure())
            if not self.broken:
                # Станок ломается, если был в рабочем состоянии
                self.process.interrupt()

# Настройка и запуск моделирования
random.seed(RANDOM_SEED)

env = simpy.Environment()
repairman = simpy.Resource(env, capacity=1)

```

```

machines = [Machine(env, f'Станок_{i + 1}', repairman) for i in
range(NUM_MACHINES)]

env.run(until=SIM_TIME)

# Результаты
s = 0
print(f'За {WEEKS} недели было выполнено:')
for machine in machines:
    s += machine.parts_made
    print(f'{machine.name}: {machine.parts_made} деталей')
print('-'*20)
print(f'Итого: {s} деталей')

```

Вывод:

```

За 4 недели было выполнено:
Станок_1: 3251 деталей
Станок_2: 3273 деталей
Станок_3: 3242 деталей
Станок_4: 3343 деталей
Станок_5: 3387 деталей
Станок_6: 3244 деталей
Станок_7: 3269 деталей
Станок_8: 3185 деталей
Станок_9: 3302 деталей
Станок_10: 3279 деталей
-----
Итого: 32775 деталей

```

4: Кинотеатр

В этой модели рассматривается процесс работы билетной кассы кинотеатра. Одновременно начинают продавать билеты на три фильма: Побег из Шоушенка, Крёстный отец, Тёмный рыцарь. На каждый из фильмов ограниченное число билетов TICKETS. Кинозрители встают в очередь и покупают билеты в случайном количестве. Если число доступных билетов меньше, чем требуется зрителю, то после небольшой дискуссии зритель покидает очередь, не купив билеты. Если после очередной покупки билетов в кассе остается не более одного билета, то считается, что билеты распроданы.

```

import random
import simpy

RANDOM_SEED = 42
TICKETS = 50 # Число билетов на фильм
SIM_TIME = 120 # Время моделирования

def moviegoer(env, movie, num_tickets, theater):
    with theater.counter.request() as my_turn:

```

```

# Подождите, пока не настанет наша очередь или пока фильм не будет
распродан
result = yield my_turn | theater.sold_out[movie]

# Проверьте, наша ли очередь или билеты на фильм распроданы
if my_turn not in result:
    theater.num_renegers[movie] += 1
    return

# Проверьте, осталось ли достаточно билетов.
if theater.available[movie] < num_tickets:
    # Кинозритель уходит после некоторого обсуждения
    yield env.timeout(0.5)
    return

# Покупка билетов
theater.available[movie] -= num_tickets
if theater.available[movie] < 2:
    # Запустите событие "распродано" для фильма
    theater.sold_out[movie].succeed()
    theater.when_sold_out[movie] = env.now
    theater.available[movie] = 0
    yield env.timeout(1)

def customer_arrivals(env, theater):
    """Генерирует новых кинозрителей"""
    while True:
        yield env.timeout(random.expovariate(1 / 0.5))

        movie = random.choice(theater.movies)
        num_tickets = random.randint(1, 6)
        if theater.available[movie]:
            env.process(moviegoer(env, movie, num_tickets, theater))

class Theater:
    def __init__(self, counter, movies, available, sold_out, when_sold_out,
num_renegers):
        self.counter = counter
        self.movies = movies
        self.available = available
        self.sold_out = sold_out
        self.when_sold_out = when_sold_out
        self.num_renegers = num_renegers

# Настройка и запуск моделирования
random.seed(RANDOM_SEED)
env = simpy.Environment()

counter = simpy.Resource(env, capacity=1)
movies = ['Побег из Шоушенка', 'Крёстный отец', 'Тёмный рыцарь']
available = {movie: TICKETS for movie in movies}
sold_out = {movie: env.event() for movie in movies}
when_sold_out = {movie: None for movie in movies}

```

```

num_renegers = {movie: 0 for movie in movies}
theater = Theater(counter, movies, available, sold_out, when_sold_out,
num_renegers)

env.process(customer_arrivals(env, theater))
env.run(until=SIM_TIME)

# Результат
for movie in movies:
    if theater.sold_out[movie]:
        print(f'Фильм {movie} был распродан за {round(theater.when_sold_out[movie])}
мин после открытия кассы')
        print(f'Число людей покинувших очередь после окончания билетов:
{theater.num_renegers[movie]}')

```

Вывод:

Фильм Побег из Шоушенка был распродан за 38 мин после открытия кассы
Число людей покинувших очередь после окончания билетов: 16
Фильм Крёстный отец был распродан за 43 мин после открытия кассы
Число людей покинувших очередь после окончания билетов: 5
Фильм Тёмный рыцарь был распродан за 28 мин после открытия кассы
Число людей покинувших очередь после окончания билетов: 5

[Ссылка](#) на блокнот Colab с реализацией рассмотренных моделей.

Задание. Вновь обратимся к задаче про абонентов мобильного оператора Мегалайн2. На занятиях 4-5 мы рассматривали три варианта вероятностных моделей для определения оптимального количества операторов и числа «мест для ожидания». Вариант 4 мы не рассматривали, т.к. он наиболее сложный с точки зрения исследования **вероятностной модели**. Однако, теперь мы можем выполнить исследование такого варианта с помощью **имитационной модели**.

Таблица 1 – Варианты математической модели обслуживания двух потоков

	Без ожидания	С ожиданием
Однородное обслуживание	Вариант 1	Вариант 2
Неоднородное обслуживание	Вариант 3	Вариант 4

Вариант 4. Call-центр мобильного оператора Мегалайн2 дает консультации своим абонентам. Звонки от абонентов тарифа Смарт поступают с интенсивностью 20 зв/час, звонки от абонентов тарифа Базовый в среднем каждые 10 минут. Среднее время консультации абонентов тарифа Смарт составляет 3 минуты, а абонентов тарифа Базовый – 7 минут. Если все операторы заняты, клиент ожидает начала соединения. Однако, число «мест для ожидания» конечно. В случае, если абоненту не хватает места и там, абонент получает отказ в консультации.

- Рассчитайте достаточное число операторов и число «мест для ожидания» такое, чтобы вероятность того, что звонок клиента будет сброшен была не более 0,01 (минимизируйте число операторов, при условии, что «мест для ожидания» может быть не более 7).

$$P\{\text{звонок будет сброшен}\} = \frac{\text{число сброшенных заявок}}{\text{общее число поступивших заявок}}$$